

docker_class_11(54)

♦ DOCKER SWARM

SERVICE (containers) → MULTIPLE SERVERS

- 👉 Runs containers across multiple machines
- 👉 Handles scaling, load balancing, self-healing

♦ DOCKER COMPOSE

MULTIPLE CONTAINERS → SINGLE SERVER

- 👉 Runs multiple services (frontend, backend, DB)
- 👉 All on one machine

♦ DOCKER STACK

MULTIPLE CONTAINERS → MULTIPLE SERVERS

♦ RELATION

DOCKER STACK = DOCKER COMPOSE + DOCKER SWARM

♦ SIMPLE MEMORY

COMPOSE = MULTIPLE CONTAINERS

SWARM = MULTIPLE SERVERS

STACK = BOTH

taken 2 servers one is m1 and another is w1

install docker and started

Docker stack is combination of compose and swarm

compose = multiple containers

swarm = multiple servers

in manger i want to install compose

```
sudo curl -L "https://github.com/docker/compose/releases/download/1.29.1/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
ls /usr/local/bin/
sudo ln -s /usr/local/bin/docker-compose /usr/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
docker-compose version
```

docker swarm init -advertise-addr private-ip

docker swarm init --advertise-addr 172.31.3.68

```
[root@ip-172-31-3-68 ~]# docker swarm init --advertise-addr 172.31.3.68
unknown shorthand flag: 'a' in --advertise-addr
See 'docker swarm init --help'.
[root@ip-172-31-3-68 ~]# docker swarm init --advertise-addr 172.31.3.68
Swarm initialized: current node (2rceirebnrgzxp088k9jknh5p) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-15yymz9acsc0twktprop05jtg6ottd7gtvj2d7hany7jco5uda-41evf6gwftrole7diktdd968z 172.31.3.68:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

taken this token and copy to worker1

```
docker swarm join --token SWMTKN-1-15yymz9acsc0twktprop05jtg6ottd7gtvj2d7hany7jco5uda-41evf6gwftrole7diktdd968z 172.31.3.68:2377
```

swarm has a worker

swarm is initialised

compose also initialised

Now we write docker stack file

vi compose.yaml

vi docker-compose.yaml

version: "3"

services:

myweb:

image: shaikmustafa/cycle

ports:

- "8081:80"

This will work with:

docker stack deploy -c docker-compose.yaml MyFirstStack

Now we are using stack, so we do not use docker compose; this has a separate command.

docker stack deploy -c docker-compose.yaml MyFirstStack

Now the stack will be created.

docker ps

You can check using docker ps.

The app can be accessed across multiple servers.

Till now we ran only a single container using compose.

Now we will run multiple containers with stack across multiple servers.

vi compose.yaml

```
version: "3"
```

```
services:
```

```
  one:
```

```
    image: shaikmustafa/dm
```

```
    ports:
```

```
      - "8081:80"
```

```
  two:
```

```
    image: shaikmustafa/cycle
```

```
    ports:
```

```
      - "8082:80"
```

```
  three:
```

```
    image: shaikmustafa/mygame
```

```
    ports:
```

```
      - "8083:80"
```

This stack needs to be deployed using:

docker stack deploy --compose-file docker-compose.yml MyApp

stack will create and we can check all services

if we learn compose it will very easy

Docker stack will not work with build it will support only image

ex: if i build image in manager it will not

if i give build path in docker file it will build only that server not others

Note: in compose build will work but where it comes to stack or swarm will use image not build

simply docker stack will not work with build it will work with only image

Docker stack advantage means we can create potrainner also

potrainner is gui tool like docker

using command we have create container and delete a container other then that through gui we can do

i.e potrainner if we want to install this we need stack and compose

Installation commands of docker potrainner gui tool

STEP 1: Update system

```
sudo apt update && sudo apt upgrade -y
```

STEP 2: Install Docker Engine

```
curl -fsSL https://get.docker.com | sudo sh
```

STEP 3: Enable and start Docker

```
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

STEP 4: Verify Docker installation

```
docker --version
```

STEP 5: Install Docker Compose plugin

```
sudo apt install docker-compose-plugin -y
```

STEP 6: Verify Docker Compose

```
docker compose version
```

STEP 7: Initialize Docker Swarm mode

```
docker swarm init
```

STEP 8: Open required firewall ports (if UFW enabled)

```
sudo ufw allow 2377/tcp
```

```
sudo ufw allow 7946/tcp
```

```
sudo ufw allow 7946/udp
```

```
sudo ufw allow 4789/udp
```

```
sudo ufw allow 9000/tcp
```

```
sudo ufw allow 9443/tcp
```

STEP 9: Download Portainer stack file

```
curl -L https://downloads.portainer.io/ce2-21/portainer-agent-stack.yml -o portainer-agent-stack.yml
```

STEP 10: Deploy Portainer stack

```
docker stack deploy -c portainer-agent-stack.yml portainer
```

STEP 11: Check stack services

```
docker stack services portainer
```

STEP 12: Check running containers

docker ps

STEP 13: Access Portainer GUI

Open browser:

http://YOUR_SERVER_PUBLIC_IP:9000

or

STEP 1: Initialize Docker Swarm

docker swarm init

STEP 2: Download latest Portainer stack

curl -L <https://downloads.portainer.io/ce2-21/portainer-agent-stack.yml> -o portainer-agent-stack.yml

STEP 3: Deploy Portainer

docker stack deploy -c portainer-agent-stack.yml portainer

STEP 4: Check services

docker service ls

STEP 5: Check stack status

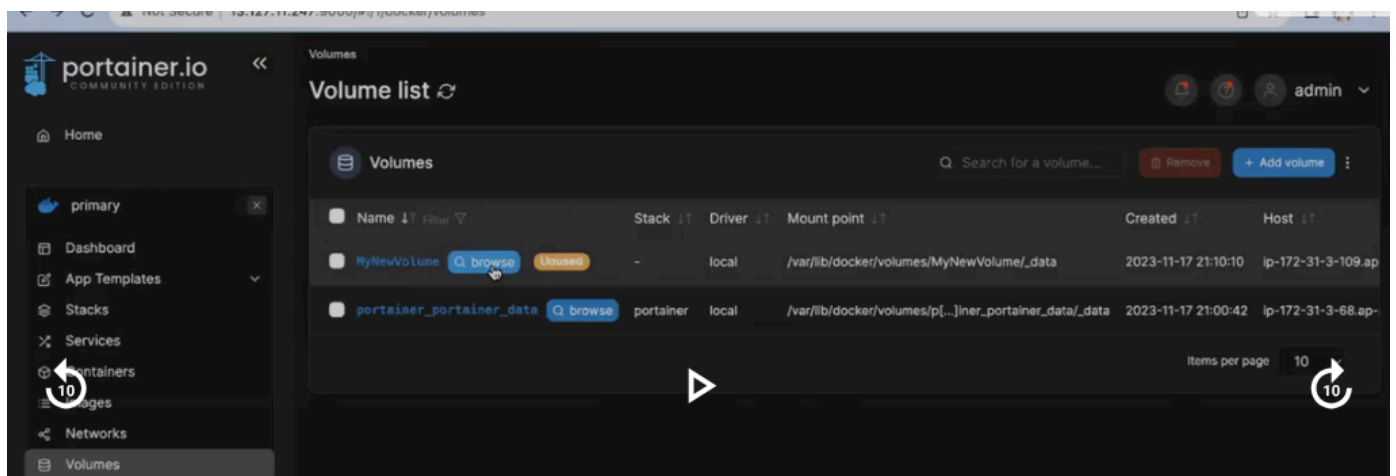
docker stack services portainer

we can watch and monitor the all

we create services in potrainer we can not remember the commands simply gui we can do

we can create containers also

and coming to volume locally if we have any files so we can easily import to volumes



In Realtime it will use only few members because it is not official docker some 3rd party it is created till now we deploy basic html now we will see java application we will deploy app in manual then later we have to automatic yum install git -y git clone <https://github.com/devops0014/one.git> cd one git

checkout master yum install java-1.8.0-openjdk -y yum install maven -y mvn clean package ll vim Dockerfile FROM tomcat:8.0.20-jre8 COPY target/*.war /usr/local/tomcat/webapps/swiggy.war save and exit docker build -t image1 . //it will pull latest save locally docker run -itd --name cont1 -p 9090:80 image1

//localhost:9090 -->click manager--it will ask credentials we have to setup to resolve this locally i will modify and push into container using copy components \\\# goto offical dlcdn folder and download apache tomcat using wget \\\# tar -zxvf apache-tomcat-9.0.83.tar.gz \\\# cd conf \\\# cp tomcat-users.xml ../. / \\\# cd ../. \\\# will edit tomcat-users.xml file will edit credentials \\\# manager gui , manager script you can get from jenkins class vedios this file vim Dockerfile FROM tomcat:8.0.20-jre8 COPY tomcat-users.xml /usr/local/tomcat/conf COPY target/*.war /usr/local/tomcat/webapps/swiggy.war docker build -t image2 . docker run -itd --name cont123 -p 2222:8080 image2

now goto application ip:2222/manager/html enter credentails if you click on swiggy the applicaion is access which is easy means docker just docker file of tomcat i.e

<https://github.com/Harishjangidi/nodejs-docker> git clone <https://github.com/Harishjangidi/nodejs-docker> ll rm Dockerfile vi Docekrfile FROM node:16 WORKDIR /app COPY package*.json ./ RUN npm install COPY . . EXPOSE 80 CMD \\\\\\\\\\\["node" , "index.js"\\\\\\\\\\\\]

docker build -t mynodeimage . docker run -itd --name cont11 -p 8888:80 mynodeimage

This we deployed node js application this image is big so we will reduce image size using stages vi Dockerfile \\\ \# THIS IS FIRST STAGE HERE APP WILL GETS DELVIRED developement part FROM node:19-alpine AS base WORKDIR /app COPY package.json ./ RUN npm install COPY . . CMD npm start

#THIS IS SECOND STAGE : APP WILL GETS DEPLOYED production part FROM base AS final RUN npm install --production COPY . . CMD \\\\\\\\\\\["node" , "index.js"\\\\\\\\\\\\]

docker build -t aws . check diff previous image and present image present 177 MB previous one 909 MB THID IS THE POWER OF MULTISTAGE DOCKER FILE IN REALTIME IF WE HAVE MORE MB USE THIS MULTISTAEG DOCKERFILE docker run -itd --name cont666 -p 6666:80 aws see now applicaion is deployed so in this way we will reduce image size

\\==DOCKER DIRECTORY DATA:== !\\[156cdf1b80effe461973f04b55e07fe5.png\\] ([_resources/83bca947a372499dba1bbba790e2173a.png](#))

my manger asked the docker we have to separeate volume i.e ebs-2

1.install jenkins start jenkins

2. will have one volume remove docker in the server remove all file

3.this server check which availabilti so based on tha we can create volume to this az

4. goto volumes vt: ssd dp2 , 100 gb , ap-south-1b avalibity zone , name: docker volume

5. this volume attach to server goto actions-> attach volume -> select ec2 --> device name /dev/xvdf

6. check how many volume of ec2

7. fdisk -l : HOW TO CHECK HOW MANY EBS IN SERVER

```
[root@ip-172-31-3-109 ~]# fdisk -l
Disk /dev/xvda: 8 GiB, 8589934592 bytes, 16777216 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: 84B5F398-58C6-4F8A-9DB3-02258ADEBC84

Device            Start      End  Sectors  Size Type
/dev/xvda1        4096 16777182 16773087   8G Linux filesystem
/dev/xvda128      2048      4095    2048    1M BIOS boot

Partition table entries are not in disk order.

Disk /dev/xvdf: 11 GiB, 11811160064 bytes, 23068672 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
[root@ip-172-31-3-109 ~]#
```

using this command we have to attach

`fdisk /dev/xvdf`

enter

n means new enter

p

1

enter

enter

particion creater

just give w it will save

`fdisk -l`

it will show partician is created

```
Device            Boot Start      End  Sectors  Size Id Type
/dev/xvdf1        2048 23068671 23066624  11G 83 Linux
[root@ip-172-31-3-109 ~]#
```

`vi /etc/fstab`

we have to add our device

`/dev/xvdf1 /var/lib/docker ext4 default 0 0`

now save and continue

`mkfs ext4 /dev/xvdf1`

`mount -a`

`df -h`

```
root@ip-172-31-3-109 ~]# mount -a
root@ip-172-31-3-109 ~]# df -h
filesystem      Size  Used Avail Use% Mounted on
devtmpfs        468M   0  468M   0% /dev
tmpfs           477M   0  477M   0% /dev/shm
tmpfs           477M 496K  476M   1% /run
tmpfs           477M   0  477M   0% /sys/fs/cgroup
/dev/xvda1      8.0G  2.5G  5.6G  31% /
tmpfs           96M   0   96M   0% /run/user/1000
tmpfs           96M   0   96M   0% /run/user/0
/dev/xvdf1      11G   24K  11G   1% /var/lib/docker
root@ip-172-31-3-109 ~]#
```

now install docker and pull one nginx image now check how much mb later

```
docker.io/library/nginx:latest
[root@ip-172-31-3-109 ~]# df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        468M   0  468M   0% /dev
tmpfs           477M   0  477M   0% /dev/shm
tmpfs           477M 508K  476M   1% /run
tmpfs           477M   0  477M   0% /sys/fs/cgroup
/dev/xvda1      8.0G  2.7G  5.4G  33% /
tmpfs           96M   0   96M   0% /run/user/1000
tmpfs           96M   0   96M   0% /run/user/0
/dev/xvdf1      11G  193M   10G   2% /var/lib/docker
[root@ip-172-31-3-109 ~]#
```